EEM 364
Mikroişlemcileri

EMU8086 ve ASSEMBLY

Assembly Dili Yapısı

[Label:] Mnemonic [Operands] [;Comment]

Döngü

Koşullar da

kullanılacak

goto

ADD AX, BX

Opcode

Destination operand

Source operand

emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

01

MAsm

FAsm

choose code template

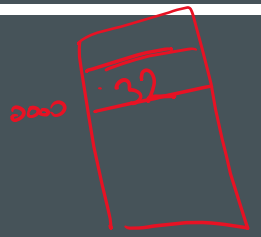
- COM template - simple and tiny executable file format, pure machine code.
- EXE template - advanced executable file. header: relocation, checksum.
- BIN template - pure binary file, allows all sorts of customizations (advanced)
- BOOT template - for creating floppy disk boot records (very advanced)

empty workspace the emulator

use Flat Assembler / Intel syntax [see: fasm_compatibility.asm in examples]

OK Cancel

Assembly Dili Yapısı: Directives



X db 32H

mov al, X

mov al, [0000]

Direktifler: Derleyici tarafından algılanan fakat I.S. içerisinde

- **Model Direktifleri**

Yazılacak olan kodun büyüklüğünü tanımlar

- **Sembol Direktifleri**

Geleneksel prog. dillerindeki değişkenlere denk gelir

[Semboller değişken veriyi içermezler]

verinin bellekteki adresini

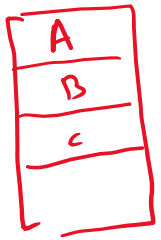
- **Segment Direktifleri**

bıraktırmayan komutlar

Derleyici de yazılan kodların düzenlenmesini sağlar

Assembly Dili Yapısı

yazi db 'ABC'



- **Karakter ve String Sabitleri**
- 'ABC' , 'X'

- **Sayısal Değerler** hex
- 26 , 23AH ~~1001b , 36q , 47d~~

decimal

~~Değişken~~ Tanımlama

DİREKTİF

- **Define Byte (DB) , Define Word (DW) ,**

sayi	db	12H
sayilar	dw	0012H , FA22H , 2B54H
harf	db	'A'
kelime	db	'selam'

sayi , sayilar…… : Sembol

Assembly Dili Yapısı: Directives

Direktifler, asm'de kod organizasyonunu sağlar. Kodların makine diline nasıl çevrileceğini belirler. Segment tanımlamaları yapılır.

Model Direktifleri:

Model Medium : Veri için max 64 kB , kod için 64 kB ve üzeri olabilir

Model Small : Tüm veri ve kodlar 64'er kB içine sığar

Model Large : Veri ve kod her ikiside 64 kB'den büyük

Assembly Dili Yapısı: Directives

Direktifler, asm'de kod organizasyonunu sağlar. Kodların makine diline nasıl çevrileceğini belirler. Segment tanımlamaları yapılır.

Segment Direktifleri (Code, Data, Stack, Extra Segment):

.code : Kod segmentinin başlangıç adresini belirler

.data : Data ||

.stack : Stack ||

Assembly Dili Yapısı: Directives

Direktifler, asm'de kod organizasyonunu sağlar. Kodların makine diline nasıl çevrileceğini belirler. Segment tanımlamaları yapılır.

CDSEG **SEGMENT**	DTSEG **SEGMENT**	STSEG **SEGMENT**
.....	.....	.....
CDSEG **ENDS**	DTSEG **ENDS**	STSEG **ENDS**

Assembly Dili Yapısı: Directives

Directives vs. Instruction Mnemonics

Data definition directives look a little like machine instruction mnemonics, but they are emphatically *not* machine instructions. One very common mistake made by beginners is looking for the binary opcode represented by a directive such as **DB** or **DW.** There is no binary opcode for **DW, DB,** and the other directives. Machine instructions, as the name implies, are instructions to the CPU itself. Directives, by contrast, are instructions to the *assembler*.

http://index-of.co.uk/Assembly/Assembly_Language_Step_by_Step_en.pdf

Assembly Dili Yapısı: Directives

```
.MODEL SMALL

Data SEGMENT

    veri db 1 , 2 , 3 , 4

Data ENDS


Code SEGMENT

Code ENDS
```

Assembly Dili Yapısı: Sembollerin Kullanımı

- **Semboller:** Verinin tutulduğu RAM bölgesinin başlangıcıdır

```
.MODEL SMALL

Data SEGMENT

    sayi db 12H , 66H , 09H

Data ENDS
```

emulator: noname.exe_

file math debug view external virtual devices virtual drive help

Load | reload | step back | single step | run | step delay ms: 0

registers

	H	L
AX	66	12
BX	00	09
CX	00	25
DX	00	00

CS	0711
IP	0014
SS	0710
SP	0000
BP	0000
SI	0000
DI	0000
DS	0710
ES	0700

0711:0014

```
07120:  8A  138  è
07121:  3E  062  >
07122:  03  003  ♥
07123:  00  000  N
07124:  F4  244  ¶
07125:  90  144  É
07126:  90  144  É
07127:  90  144  É
07128:  90  144  É
07129:  90  144  É
0712A:  90  144  É
0712B:  90  144  É
0712C:  90  144  É
0712D:  90  144  É
0712E:  90  144  É
```

0711:0014

```
MOV AX, 00710h
MOV DS, AX
MOV AL, [00000h]
MOV AH, [00001h]
MOV BL, [00002h]
MOV BH, [00003h]
HLT
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
```

screen | source | reset | aux | vars | debug | stack | flags

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 12H , 66H , 09H

Data ENDS


Code SEGMENT

mov ax , @Data
mov ds , ax

mov al , sayi
mov ah , sayi+1
mov bl , sayi+2
mov bh , sayi+3

hlt

Code ENDS
```

```
.MODEL SMALL

Data SEGMENT

    sayi db 12H , 66H , 09H

Data ENDS
```

emulator: noname.exe_

file math debug view external virtual devices virtual drive help

Load | reload | step back | single step | run | step delay ms: 0

registers

	H	L
AX	66	12
BX	00	09
CX	00	25
DX	00	00
CS	0711	
IP	0014	
SS	0710	
SP	0000	
BP	0000	
SI	0000	
DI	0000	
DS	0710	
ES	0700	

0711:0014

```
07120: 8A 138 è
07121: 3E 062 >
07122: 03 003 ♥
07123: 00 000 NU
07124: F4 244 ¶
07125: 90 144 É
07126: 90 144 É
07127: 90 144 É
07128: 90 144 É
07129: 90 144 É
0712A: 90 144 É
0712B: 90 144 É
0712C: 90 144 É
0712D: 90 144 É
0712E: 90 144 É
```

0711:0014

```
MOV AX, 00710h
MOV DS, AX
MOV AL, [00000h]
MOV AH, [00001h]
MOV BL, [00002h]
MOV BH, [00003h]
HLT
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
```

screen | source | reset | aux | vars | debug | stack | flags

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 12H , 66H , 09H

Data ENDS


Code SEGMENT

mov ax , @Data
mov ds , ax

mov  al , sayi[0]
mov  ah , sayi[1]
mov  bl , sayi[2]
mov  bh , sayi[3]

hlt

Code ENDS
```

```
.MODEL SMALL

Data SEGMENT

    sayi db 12H , 66H , 09H

Data ENDS
```

emulator: noname.exe_

file math debug view external virtual devices virtual drive help

Load | reload | step back | single step | run | step delay ms: 0

registers

	H	L
AX	66	12
BX	00	09
CX	00	26
DX	00	00
CS	0711	
IP	0015	
SS	0710	
SP	0000	
BP	0000	
SI	0000	
DI	0000	
DS	0710	
ES	0700	

0711:0015

```
07120: 8A 138 è
07121: 7C 124 |
07122: 03 003 ♥
07123: 8B 139 ï
07124: D6 214 f
07125: F4 244 ¶
07126: 90 144 É
07127: 90 144 É
07128: 90 144 É
07129: 90 144 É
0712A: 90 144 É
0712B: 90 144 É
0712C: 90 144 É
0712D: 90 144 É
0712E: 90 144 É
```

0711:0015

```
MOV AX, 00710h
MOV DS, AX
MOV SI, 00000h
MOV AL, [SI]
MOV AH, [SI] + 01h
MOV BL, [SI] + 02h
MOV BH, [SI] + 03h
MOV DX, SI
HLT
NOP
NOP
NOP
NOP
NOP
...
```

screen | source | reset | aux | vars | debug | stack | flags

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 12H , 66H , 09H

Data ENDS


Code SEGMENT


mov ax , @Data
mov ds , ax

lea si , sayi
mov al , [si]
mov ah , [si+1]
mov bl , [si+2]
mov bh , [si+3]
mov dx , si
hlt

Code ENDS
```

2266

```
.MODEL SMALL

Data SEGMENT

    sayi db 12H , 66H , 09H

Data ENDS
```

emulator: noname.exe_

file math debug view external virtual devices virtual drive help

Load | reload | step back | single step | run | step delay ms: 0

registers

	H	L
AX	66	12
BX	00	09
CX	00	26
DX	00	00
CS	0711	
IP	0015	
SS	0710	
SP	0000	
BP	0000	
SI	0000	
DI	0000	
DS	0710	
ES	0700	

0711:0015

```
07120: 8A 138 è
07121: 7C 124 |
07122: 03 003 ♥
07123: 8B 139 ï
07124: D6 214 f
07125: F4 244 ¶
07126: 90 144 É
07127: 90 144 É
07128: 90 144 É
07129: 90 144 É
0712A: 90 144 É
0712B: 90 144 É
0712C: 90 144 É
0712D: 90 144 É
0712E: 90 144 É
```

0711:0015

```
MOV AX, 00710h
MOV DS, AX
MOV SI, 00000h
MOV AL, [SI]
MOV AH, [SI] + 01h
MOV BL, [SI] + 02h
MOV BH, [SI] + 03h
MOV DX, SI
HLT
NOP
NOP
NOP
NOP
NOP
...
```

screen | source | reset | aux | vars | debug | stack | flags

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 12H , 66H , 09H

Data ENDS


Code SEGMENT


mov ax , @Data
mov ds , ax


mov si , offset sayi
mov al , [si]
mov ah , [si+1]
mov bl , [si+2]
mov bh , [si+3]
mov dx , si
hlt

Code ENDS
```

Assembly Dili Yapısı: Sembollerin Kullanımı

- **Semboller:** Verinin tutulduğu RAM bölgesinin başlangıcıdır.

- LEA ve OFFSET kullanımı ile adres değeri yazmaç içine çekilebilir

LeA ⇒ Sembollerin adres bilgisini bir yazmaç içerisine aktarır

Assembly Dili Yapısı: Sembollerin Kullanımı

- **Semboller:** Verinin tutulduğu RAM bölgesinin başlangıcıdır.

- LEA ve OFFSET kullanımı ile adres değeri yazmaç içine çekilebilir

- veri db 5 DUP(12H) , veri db 5 DUP(12H,5AH)

veri

12
12
12

12
5A
12
5A
⋮

5 tane

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 5 DUP(12H)

Data ENDS


Code SEGMENT
```

Random Access Memory

0710:0000 update table li

```
0710:0000  12 12 12 12 12 00
0710:0010  B8 10 07 8E D8 90
0710:0020  90 90 90 90 90 90
0710:0030  00 00 00 00 00 00
0710:0040  00 00 00 00 00 00
0710:0050  00 00 00 00 00 00
0710:0060  00 00 00 00 00 00
0710:0070  00 00 00 00 00 00
```

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 5 DUP (12H , 2FH)

Data ENDS


Code SEGMENT
```

Random Access Memory

0710:0000 update table list

```
0710:0000  12 2F 12 2F 12 2F 12 2F-12 2F
0710:0010  B8 10 07 8E D8 90 90 90-90 90
0710:0020  90 90 90 90 90 90 90 90-90 F4
0710:0030  00 00 00 00 00 00 00 00-00 00
0710:0040  00 00 00 00 00 00 00 00-00 00
0710:0050  00 00 00 00 00 00 00 00-00 00
0710:0060  00 00 00 00 00 00 00 00-00 00
0710:0070  00 00 00 00 00 00 00 00-00 00
```

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 5 DUP("Selam")

Data ENDS


Code SEGMENT
```

Random Access Memory

0710:0000 | update | table | list

```
0710:0000  53 65 6C 61 6D 53 65 6C-61 6D 53 65 6C 61 6D 53    SelamSelamSelamS
0710:0010  65 6C 61 6D 53 65 6C 61-6D 00 00 00 00 00 00 00    elamSelam.......
0710:0020  B8 10 07 8E D8 90 90 90-90 90 90 90 90 90 90 90    ©►.ÄÏÉÉÉÉÉÉÉÉÉÉ
0710:0030  90 90 90 90 90 90 90 90-90 F4 00 00 00 00 00 00    ÉÉÉÉÉÉÉÉÉ¶......
0710:0040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00    ................
0710:0050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00    ................
0710:0060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00    ................
0710:0070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00    ................
```

```
.MODEL SMALL

Data SEGMENT

sayi    dw 5 DUP(01H)


Data ENDS
```

00 01

Random Access Memory

0710:0000 update table list

```
0710:0000  01 00 01 00 01 00 01 00-01 00
0710:0010  90 90 90 90 90 90 90 90-90 90
0710:0020  00 00 00 00 00 00 00 00-00 00
0710:0030  00 00 00 00 00 00 00 00-00 00
0710:0040  00 00 00 00 00 00 00 00-00 00
0710:0050  00 00 00 00 00 00 00 00-00 00
0710:0060  00 00 00 00 00 00 00 00-00 00
0710:0070  00 00 00 00 00 00 00 00-00 00
```

SI:

original source code

```
.MODEL SMALL

Data SEGMENT

veri dw 5 dup(02H , 03H)

Data ENDS


Code SEGMENT

Code ENDS
```

Random Access Memory

0710:0000 update table list

```
0710:0000  02 00 03 00 02 00 03 00-02 00 03 00 02 00 03 00
0710:0010  02 00 03 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0020  90 90 90 90 90 90 90 90-90 90 90 90 90 90 90 90
0710:0030  90 90 90 90 F4 00 00 00-00 00 00 00 00 00 00 00
0710:0040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Assembly Dili Yapısı: Sembollerin Kullanımı

- **BYTE PTR ,WORD PTR**

RAM bölgesinden ne kadarlık veri çekileceğini belirler

in Kullanımı

original source code

```
.MODEL SMALL


Data SEGMENT


sayi db 5 DUP(12H , 3FH , 1EH)


Data ENDS


Code SEGMENT
```

Random Access Memory

0710:0000 update table list

```
0710:0000  12 3F 1E 12 3F 1E 12 3F-1E 12 3F 1E 12 3F 1E 00
0710:0010  B8 10 07 8E D8 90 90 90-90 90 90 90 90 90 90 90
0710:0020  90 90 90 90 90 90 90 90-90 F4 00 00 00 00 00 00
0710:0030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

```
.MODEL SMALL

Data SEGMENT

    sayi db 5 DUP(12H , 3FH , 1EH)
    veri dw 53FFH

Data ENDS
```

emulator: noname.exe_

file math debug view external virtual devices virtual drive help

Load | reload | step back | single step | run | step delay ms: 0

registers

	H	L
AX	07	FF
BX	00	00
CX	00	2F
DX	00	00
CS	0712	
IP	000F	
SS	0710	
SP	0000	
BP	0000	
SI	0000	
DI	0000	
DS	0710	
ES	0700	

0712:000F

```
0712F: 90 144 É
07130: 90 144 É
07131: 90 144 É
07132: 90 144 É
07133: 90 144 É
07134: 90 144 É
07135: 90 144 É
07136: 90 144 É
07137: 90 144 É
07138: 90 144 É
07139: 90 144 É
0713A: 90 144 É
0713B: 90 144 É
0713C: 90 144 É
0713D: 90 144 É
```

0712:000B

```
MOV AX, 00710h
MOV DS, AX
MOV w.[00000h], 00044h
MOV AL, [0000Fh]
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
```

screen | source | reset | aux | vars | debug | stack | flags

original source code

```
.MODEL SMALL

Data SEGMENT

sayi db 5 DUP(12H , 3FH , 1EH)
veri dw 53FFH

Data ENDS


Code SEGMENT

mov ax , @Data
mov ds , ax

mov word ptr sayi , 44H
mov al , byte ptr veri
```

Random Access Memory

0710:0000 | update | table | list

```
0710:0000  44 00 1E 12 3F 1E 12 3F-1E 12 3F 1E 12 3F 1E FF
0710:0010  53 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0020  B8 10 07 8E D8 C7 06 00-00 44 00 8A 06 0F 00 90
0710:0030  90 90 90 90 90 90 90 90-90 90 90 90 90 90 90 90
0710:0040  90 90 90 F4 00 00 00 00-00 00 00 00 00 00 00 00
0710:0050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0710:0070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
```

Assembly Dili Yapısı: Sembollerin Kullanımı

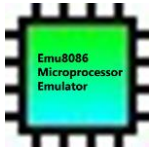
- **Semboller:** Verinin tutulduğu RAM bölgesinin başlangıcıdır.
- LEA ve OFFSET kullanımı ile adres değeri yazmaç içine çekilebilir
- veri db 5 DUP(12H) , veri db 5 DUP(12H,5AH)
- **BYTE PTR ,WORD PTR**

Assembly Dili Yapısı: Etiketler ve Döngüler

```
.MODEL SMALL

Data SEGMENT

x db 01H , 02H , 03H , 04H , 05H

Data ENDS

Code SEGMENT
mov ax , @Data
mov ds , ax

Code ENDS
```

→ Dışarıdan alınan verilerin toplanması (byte)

→ Sayaç Eklenmesi



Assembly Dili Yapısı: Etiketler ve Döngüler

- Fibonacci Serisi

```
mov ax , 0
mov bl , 0

mov al , 0
mov ah , 1

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl
```

Assembly Dili Yapısı: Etiketler ve Döngüler

- **Etiket:** Yüksek seviye programlama dillerinde **goto** kullanımına benzer. Dallanma komutlarıyla çalışırlar.

Assembly Dili Yapısı: Etiketler ve Döngüler

Jump instructions that test single flag

Instruction	Description	Condition	Opposite Instruction
JZ , JE	Jump if Zero (Equal).	ZF = 1	JNZ, JNE
JC , JB, JNAE	Jump if Carry (Below, Not Above Equal).	CF = 1	JNC, JNB, JAE
JS	Jump if Sign.	SF = 1	JNS
JO	Jump if Overflow.	OF = 1	JNO
JPE, JP	Jump if Parity Even.	PF = 1	JPO
JNZ , JNE	Jump if Not Zero (Not Equal).	ZF = 0	JZ, JE
JNC , JNB, JAE	Jump if Not Carry (Not Below, Above Equal).	CF = 0	JC, JB, JNAE
JNS	Jump if Not Sign.	SF = 0	JS
JNO	Jump if Not Overflow.	OF = 0	JO

http://www1.frm.utn.edu.ar/arquitectura/t86.pdf

Assembly Dili Yapısı: Etiketler ve Döngüler

Jump instructions that test single flag

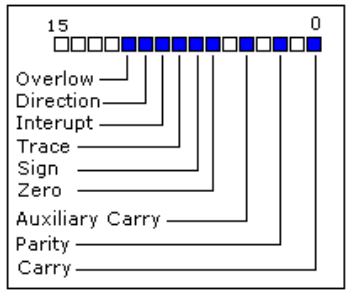
Instruction	Description	Condition	Opposite Instruction
JZ , JE	Jump if Zero (Equal).	ZF = 1	JNZ, JNE
JC , JB, JNAE	Jump if Carry (Below, Not Above Equal).	CF = 1	JNC, JNB, JAE
JS	Jump if Sign.	SF = 1	JNS
JO	Jump if Overflow.	OF = 1	JNO
JPE, JP	Jump if Parity Even.	PF = 1	JPO
JNZ , JNE	Jump if Not Zero (Not Equal).	ZF = 0	JZ, JE
JNC , JNB, JAE	Jump if Not Carry (Not Below, Above Equal).	CF = 0	JC, JB, JNAE
JNS	Jump if Not Sign.	SF = 0	JS
JNO	Jump if Not Overflow.	OF = 0	JO

http://www1.frm.utn.edu.ar/arquitectura/t86.pdf

8086 assembler tutorial for beginners (part 6)

Arithmetic and Logic Instructions

Most Arithmetic and Logic Instructions affect the processor status register (or **Flags**)



ADD, SUB,CMP, AND, TEST, OR, XOR

Assembly Dili Yapısı: Etiketler ve Döngüler

```
.MODEL SMALL

Data SEGMENT

x db 01H , 02H , 03H , 04H , 05H
y db 0

Data ENDS

Code SEGMENT

mov ax , @Data
mov ds , ax

mov ax , 0
mov di , 5
mov si , 0

TPL: add ah , x[si]
     inc si
     dec di
     jnz TPL

mov y , ah

hlt

Code ENDS
```

- Dizilerin toplanması (byte)

Etiketler ve Döngüler

```
.MODEL SMALL

Data SEGMENT

x db 01H , 02H , 03H , 04H , 05H
y db 0

Data ENDS

Code SEGMENT

mov ax , @Data
mov ds , ax

mov ax , 0
mov di , 5
mov si , 0

TPL: add ah , x[si]
     inc si
     dec di
     jnz TPL

mov y , ah

hlt

Code ENDS
```

- Dizilerin toplanması (word)

```
01 00 02 00 03 00 04 0
BB 10 07 BF DB B9 05 0
```

LAB Ödevi-1

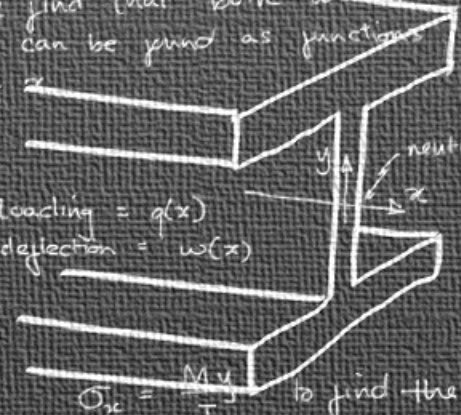
- X db 1,2,3,4,5,6,7,8,9

Tek indisliler bir yerde çift indisliler bir yerde toplansın

İpucu: di ve si iki indis yazmacı kullanılır

- Fibonacci serisi: Etiket kullanarak

given a beam of constant EI
we find that both w and
q can be found as functions
of x



loading = $q(x)$
deflection = $w(x)$

$\sigma_x = \frac{My}{I}$ to find the tension force at some location y

for a static system

$$\frac{\partial^2}{\partial x^2}\left(EI\frac{\partial^2 w}{\partial x^2}\right) = q$$

for $\frac{\partial EI(x)}{\partial x} = 0$

where $\frac{\partial^4 w}{\partial x^4} EI = q(x)$

E = Young's Modulus

I = Second moment of area

http://bsnscb.com/data/out/124/39800806-maths-wallpapers.jpg